

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

NAME:

USN:

SEM/ SEC:

TERM: 7th October 2024 to 25th January 2025

Co-ordinator: Dr. M. Sinthuja

I / II SEMESTER

BASICS OF JAVA PROGRAMMING

Course Code: PLC143

Course Credits: 2:0:1

COURSE DESIGN, DELIVERY AND ASSESSMENT

Course Code and Title: PLC143- Basics of Java Programming	Course Credit: 2:0:1
CIE: 50M	SEE: 50 Marks
Total No of Lab Sessions/ Hours: 14 sessions / 28 hours	

COURSE OBJECTIVES:

1. Learn the fundamental features of object-oriented programming
2. Understand the concepts of operators and java programming constructs.
3. Discover the concepts of class creation and methods
4. Explore the principles of inheritance and abstraction
5. Examine the fundamentals of importing packages and exception handling techniques

COURSE CONTENT:

1. Java Program to find the largest using simple if and smallest of three numbers using a ternary operator.
2. Java program to generate fibonacci series, using while and for loop.
3. Java program
 - a. To calculate a factorial of a number
 - b. To check the number is palindrome.
4. Java program to demonstrate the break statement with respect to exit from the loop and the “civilized” form of goto.
5. Java Program to print the largest element in an array.
6. Java Program to add two matrices
7. Java program to find the largest element of an array and search the given key element from the array.
8. Java program to add two complex numbers using class.
9. Java class called Student with the following details as variables within it. USN, NAME, BRANCH, PHONE, PERCENTAGE, write a java program to create n student objects and print the USN, Name, Branch, Phone, and percentage of these objects with suitable headings.
10. Java program demonstrating method overloading and constructor overloading.
11. Java program to demonstrate the uses of super. Design a super class called Staff with details as StaffId, Name, Salary. Extend this class by writing two subclasses namely teaching (expertise), and technical (skills), Write a java program to read and display staff details of all two categories.
12. Describe the abstract class called shape, which has three subclasses say triangle, rectangle and circle. Define one method area () in abstract class and override this area () in these three subclasses to calculate areas of triangle, rectangle and circle.
13. Java program to read two integers a and b. Compute a/b and print, when b is not zero. Raise an exception when b is equal to zero. Also demonstrate working of ArrayIndexOutOfBoundsException.
14. Java program demonstrating finally block for handling exceptions.
15. Java program to implement multiple inheritance.

SUGGESTED LEARNING RESOURCES:

Reference Books/ Web Links:

1. Herbert Schildt, Java The Complete Reference, 7th Edition, Tata McGraw Hill, 2007.
2. https://onlinecourses.nptel.ac.in/noc22_cs47/preview

Course outcome:

At the end of the course, students will be able to:

CO1: Understand object oriented programming concepts (PO 1)

CO2: Apply Java programming constructs to solve the given problem. (PO 1,2, 3)

CO3: Apply the concept of classes and objects to solve the given problem. (PO 1,2, 3)

CO4: Use the concepts of polymorphism and inheritance to solve the given problem. (PO 1,2, 3) CO5:

Design solutions for the given real world problems using concepts of packages, interfaces and Exception Handling. (PO 1, 2, 3)

COURSE ASSESSMENT AND EVALUATION:

Continuous Internal Evaluation (CIE): 50 Marks		
Assessment Tool	Marks	Course outcomes addressed
Internal test-I	30	CO1, CO2
Internal test-II	30	CO3, CO4, CO5
Average of the two internal tests shall be taken for 30 marks.		
Other components		
Lab Component Evaluation	20	CO1, CO2, CO3, CO5, CO5
Semester-End Examination (SEE)	100	CO1, CO2, CO3, CO4, CO5

GUIDELINES TO STUDENTS IN LAB

1. Disciplinary to be maintained by the students in the Lab
2. Students are required to carry their lab observation book and record book with completed experiments while entering the lab.
3. Students must use the equipment with care. Any damage caused by the student is punishable.
4. Students are not allowed to use their cell phones/pen drives/ CDs in labs.
5. Students need to be maintain proper dress code along with ID Card
6. Students are supposed to occupy the computers allotted to them and are not supposed to talk or make noise in the lab.
7. Students, after completion of each experiment they need to be updated in observation notes and same to be updated in the record.
8. Lab records need to be submitted after completion of the experiment and get it corrected with the concerned lab faculty.
9. If a student is absent for any lab, they need to complete the same experiment in their free time before attending the next lab.

STEPS TO PERFORM EXPERIMENTS IN THE LAB BY THE STUDENT

1. Students have to write the date, aim and problem statement for the given experiment in the record book.
2. Students have to listen and understand the experiment explained by the faculty and note down the important points in the book.
3. Analyse and Develop/implement the logic of the program by the student in respective platforms.
4. After approval of the logic of the experiment by the faculty, the experiment has to be executed.
5. Debug the programs and introduce errors in the program and analyse the errors.
6. After successful execution the results are to be shown to the faculty and noted the same in the record book.
7. Students need to attend the Viva-Voce on that experiment.
8. Update the completed experiment in the record and submit it to the concerned faculty in-charge.

GENERAL LABORATORY INSTRUCTIONS

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before the commencement, come prepared to the lab with the synopsis / program / experiment details.
3. Students should enter into the laboratory with: a. Laboratory Record updated up to the last session experiments and other utensils (if any) needed in the lab. b. Proper Dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results / output in the lab record note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, and must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high end branded systems, which should be utilized properly.
8. Students must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviours with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out; if anybody found loitering outside the lab / class without permission during working hours will be treated seriously and punished appropriately.
10. Students should LOG OFF/ SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system / seat is kept properly.

EVALUATION OF JAVA PROGRAMS

S.No	List of Programs	Date of Execution	Marks Obtained (10M)	Faculty Signature
1	Java Program to find the largest using simple if and smallest of three numbers using a ternary operator.			
2	Write a Java program to generate Fibonacci series, using while and for loop.			
3	Write a Java program - To calculate a factorial of a number - To check the number is palindrome.			
4	Write a JAVA program to demonstrate the break statement with respect to exit from the loop and the “civilized” form of goto.			
5	Java Program to print the largest element in an array.			
6	Java Program to Add Two Matrices.			
7	Write a Java program to find the largest element of an array and search the given key element from the array.			
8	Write a JAVA program to add two complex numbers using Class.			
9	Create a JAVA class called Student with the following details as variables within it. USN, NAME, BRANCH, PHONE, PERCENTAGE, write a JAVA program to create n Student objects and print the USN, Name, Branch, Phone, and percentage of these objects with suitable headings.			
10	Write a JAVA program demonstrating Method overloading and Constructor overloading.			
11	Java Program to demonstrate the uses of super. Design a super class called Staff with details as StaffId, Name, Salary. Extend this class by writing two subclasses namely Teaching (Expertise), and Technical (skills), Write a JAVA program to read and display staff details of all two categories.			
12	Describe the abstract class called shape, which has three subclasses say triangle, rectangle and circle. Define one method area () in abstract class and override this area () in these three subclasses to calculate areas of triangle, rectangle and circle.			
13	Write a JAVA program to read two integers a and b. Compute a/b and print, when b is not zero. Raise an exception when b is equal to zero. Also demonstrate working of ArrayIndexOutOfBoundsException.			

14	Write a JAVA program demonstrating finally block for handling exceptions.			
15	Java program to implement multiple inheritance.			

INTRODUCTION TO JAVA PROGRAMMING

INTRODUCTION

Java is a class-based, object-oriented programming language. The language is intended to have the fewest possible implementation dependencies. The goal of this language is to relieve developers of the burden of creating code for each platform. With this language, the phrase "write once and run everywhere," or "WORA," is frequently used. This implies that when we compile a Java code, we obtain the byte code (.class file), which may be run on many systems that accept Java without requiring another compilation. The Java language was created in 1995. The primary uses for it are in desktop, mobile, and web development. The stability, security, and simplicity of the Java language are well-known. It is intended to require the least amount of implementation.

History

There is a fascinating history behind the Java language. The Green team, consisting of Patrick Naughton, Mike Sheridan, and James Gosling, began developing Java in 1991. These individuals worked at Sun Microsystems as engineers. The first public implementation, known as Java 1.0, was made available in 1996. Arthur Van Hoff rewrote the Java 1.0 compiler to adhere closely to its specifications. Since Java 2 was released, several configurations for different platforms have been included in the latest versions of the software. It's important to remember that James Gosling is regarded as the founder of Java.

OOPs (Object-Oriented Programming System)

Object Oriented Programming (OOP) is a programming paradigm that focuses on the use of objects to represent and manipulate data. In OOP, data is encapsulated within objects, and objects are defined by their properties (attributes) and behaviors (methods). OOP provides several key concepts that enable developers to write modular, reusable, and maintainable code.

The concepts are:

- **Object**
- **Class**
- **Inheritance**
- **Polymorphism**
- **Abstraction**
- **Encapsulation**

Class

Collection of objects is called class. It is a logical entity.

A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

Object

Any entity that has state and behaviour is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class.

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

If one task is performed in different ways, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example, phone call, we don't know the internal processing.

Encapsulation

Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines.

Structure of class

```
class Name_of_the_class
{
    //Data

    //Functions
}
```

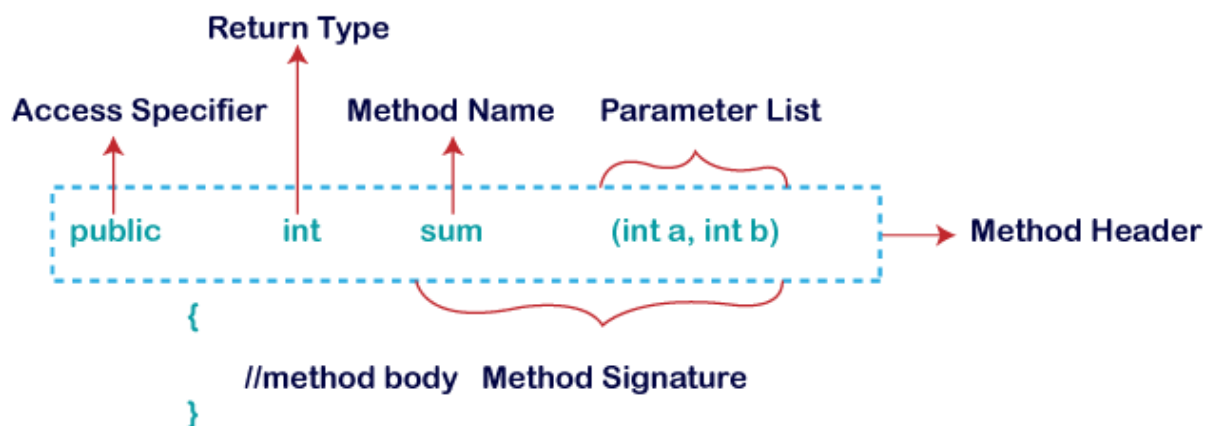
Basic Program

```
class MyFirstJavaProgram
{
    public static void main(String[] args)
    {
        System.out.println("Hello, World!");
    }
}
```

Methods in Java

A method is a series of statements, a block of code, or a collection of code that are combined to carry out a certain function.

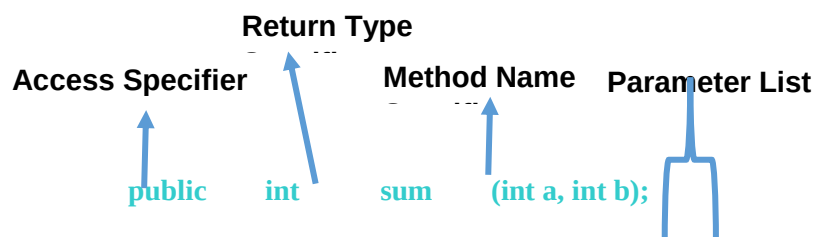
Method Definition Syntax



Method Declaration

The method declaration defines all the method's attributes, such as access level, return type, name, and arguments as shown below. The method body is where all the action takes place. It contains the instructions that implement the method.

Syntax



Object Syntax:

Using the new keyword to create an object of the class `class_name`, we can also see how the new keyword is used to invoke the class's constructor.

`class_name object_name = new class_name();`

Creating Program:

Step 1: Use text editor (gedit) or IDE

Step 2: Java source code file name must end with `program_name.java`

Compiling and Executing Java Program

C:\> javac MyFirstJavaProgram.java

C:\> java MyFirstJavaProgram

- Popular IDE's are NetBeans, Eclipse, IntelliJ

STEPS TO WRITE AND EXECUTE JAVA PROGRAMS USING THE VI EDITOR

Step-by-Step Procedure

1. *Open Terminal (Command Line Interface)*
2. *Create a Java Program Using vi Editor*

- To write a Java program, you need to create a .java file using the vi editor.

To create and open a new file:

```
vi ProgramName.java
```

- This will open the vi editor and create a new file named ProgramName.java.

3. Write Java Code

- In vi editor, the default mode is **command mode**. To enter **insert mode** (to write code), press **i** or **insert** key.
- After writing your Java code, you need to save and exit vi:
 - Press **Esc** to go back to **command mode**.
 - Type **:wq** and press **Enter**. This command writes the file and quits the editor.

4. Compile the Java Program

- Once the program is saved, return to the terminal (if you're not already there) and compile the Java program using the `javac` compiler.

To compile:

```
javac ProgramName.java
```

- If there are no syntax errors, this will create a ProgramName.class file, which is the bytecode version of your Java program.
- If there are errors, they will be displayed in the terminal. You must fix these errors in vi and save the file again.

5. Run the Java Program

- Once the program is compiled, you can run the Java program using the `java` command.

To run the compiled program:

```
java ProgramName
```

- This will produce output:

PRACTICE PROGRAMS

Example 1: Program to print Hello world

```
class HelloWorld
{
    public static void main (String args[])
    {
        System.out.println("Hello World");
    }
}
```

Example 2: Program to add two numbers

```
public class JavaExample {
    public static void main(String[] args) {
        // two integer variables with values
        // and a variable "sum" to store the result
        int num1 = 5, num2 = 15, sum;
        //calculating the sum of num1 and num2 and
        //storing the result in the variable sum
        sum = num1+num2;
        //printing the result
        System.out.println("Sum of "+num1+" and "+num2+" is: "+sum);
    }
}
```

Example 3: Java Program to demonstrate the example of for loop

Syntax:

```
public class ForExample
{
    public static void main(String[] args)
    {
        //Code of Java for loop
        for(int i=1;i<=10;i++)
        {
```

```

        System.out.println(i);
    }
}

```

Example 4: Java Nested for Loop

```

public class NestedForExample
{
    public static void main(String[] args)
    {
        //loop of i
        for(int i=1;i<=3;i++)
        {
            //loop of j
            for(int j=1;j<=3;j++)
            {
                System.out.println(i+" "+j);
            }
        }
    }
}

```

Example 5 :Program to print Pyramid of numbers

```

public class PyramidExample
{
    public static void main(String[] args)
    {
        for (int i=1; i<=5; i++)
        {
            for (int j=1; j<=i; j++)
            {
                System.out.println("* ");
            }
        }
    }
}

```

```

        }
        System.out.println(); //new line
    }
}
}

```

Example 6: Program to print Pyramid of *

```

public class PyramidExample2
{
    public static void main (String [] args)
    {
        int term=6;
        for (int i=1; i<=term; i++)
        {
            for (int j=term; j>=i; j--)
            {
                System.out.println("* ");
            }
            System.out.println(); //new line
        }
    }
}

```

Example 7: Java While Loop

```

public class WhileExample
{
    public static void main(String[] args)
    {
        int i=1;
        while(i<=10)
        {
            System.out.println(i);
            i++;
        }
    }
}

```

```

        }
    }
}

```

Example 8: Java Do While Loop

```

public class DoWhileExample
{
    public static void main(String[] args)
    {
        int i=1;
        do
        {
            System.out.println(i);
            i++;
        }
        while(i<=10);
    }
}

```

Example 9: Java Break Statement with Loop

//Java Program to demonstrate the use of break statement

//inside the for loop.

```

public class Break Example
{
    public static void main (String [] args)
    {
        //using for loop for (int i=1; i<=10; i++)
        {
            if(i==5)

```

```

    {
        //breaking the loop break;
    }
    System.out.println(i);
}
}

```

Example 10: Java Continue Statement

//Java Program to demonstrate the use of a continue statement inside the for loop.

```

public class ContinueExample
{
    public static void main (String [] args)
    {
        //for loop for (int i=1; i<=10; i++)
        {
            if(i==5)
                continue;
            //it will skip the rest statement
            System.out.println(i);
        }
    }
}

```

SESSION 1: JAVA PROGRAM ON IF AND TERNARY OPERATOR

Write and execute a Java Program to find the largest using simple if and smallest of three numbers using a ternary operator

Algorithm: To Find the Largest of Three Numbers Using if:

1. Start.
2. Input three numbers: a, b, c.
3. Initialize a variable largest.
4. Compare a with b and c using simple if conditions:
 - If a is greater than or equal to b and a is greater than or equal to c, then assign largest = a.
 - Else, check if b is greater than or equal to a and b is greater than or equal to c. If true, assign largest = b.
 - Otherwise, assign largest = c.
5. Output the value of largest.
6. End.

Algorithm: To Find the Smallest of Three Numbers Using a Ternary Operator:

1. Start.
2. Input three numbers: a, b, c.
3. Use a nested ternary operator to find the smallest:
 - First, compare a and b to find the smaller one.
 - Compare the smaller of a and b with c to get the smallest of all three.
4. Store the result in a variable smallest.
5. Output the value of smallest.
6. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 2: JAVA PROGRAM ON WHILE AND FOR LOOP

Write and execute a Java program to find Fibonacci series, using while and for loop.

Algorithm: To Find Fibonacci Series Using a while Loop:

1. Start.
2. Input the number of terms n for the Fibonacci series.
3. Initialize three variables:
 - $a = 0$ (First Fibonacci number)
 - $b = 1$ (Second Fibonacci number)
 - $count = 0$ (Counter to track the current term number)
4. Print the first two terms (a and b) if n is greater than or equal to 2.
5. Use a while loop:
 - Check if $count$ is less than $n - 2$ (since the first two terms are already printed).
 - Compute the next term as $c = a + b$.
 - Print the value of c .
 - Update $a = b$ and $b = c$ to move to the next terms.
 - Increment $count$ by 1.
6. End the loop when the counter reaches $n - 2$.
7. End.

Algorithm to Find Fibonacci Series Using a for Loop:

1. Start.
2. Input the number of terms n for the Fibonacci series.
3. Initialize two variables:
 - $a = 0$ (First Fibonacci number)
 - $b = 1$ (Second Fibonacci number)
4. Print the first two terms (a and b) if n is greater than or equal to 2.
5. Use a for loop:
 - Loop from $i = 2$ to $n - 1$ (since the first two terms are already printed).
 - Compute the next term as $c = a + b$.
 - Print the value of c .
 - Update $a = b$ and $b = c$ to move to the next terms.
6. End the loop after $n - 2$ iterations.
7. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 3: JAVA PROGRAM ON FACTORIAL and PALINDROME

Write a Java program

- a. To calculate a factorial of a number
- b. To check the number is palindrome.

Algorithm. To Calculate the Factorial of a Number

1. Start.
2. Input a number n.
3. Initialize a variable factorial = 1.
4. If n is 0 or 1, the factorial is 1 (special case for factorial definition).
5. Otherwise, use a loop to calculate the factorial:
 - o For every number i from 2 to n (inclusive):
 - Multiply factorial by i.
6. Output the value of factorial.
7. End.

Algorithm. To Check if a Number is a Palindrome

1. Start.
2. Input a number n.
3. Convert the number to a string for easy reversal.
4. Reverse the string representation of the number.
5. Compare the original string and the reversed string:
 - o If they are the same, the number is a palindrome.
 - o Otherwise, it is not a palindrome.
6. Output the result.
7. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 4: JAVA PROGRAM ON STATEMENTS

Write a JAVA program to demonstrate the break statement with respect to exit from the loop and the “civilized” form of goto.

Algorithm: Using break to Exit a Loop

1. Start.
2. Initialize or input the data required for the loop (e.g., a counter or list of values).
3. Start a loop (e.g., while or for loop).
4. Within the loop:
 - Check for a specific condition that would require exiting the loop.
 - If the condition is met, execute the break statement to exit the loop.
5. If the break statement is not executed, the loop continues until its natural termination.
6. After exiting the loop, perform any necessary actions or output results.
7. End.

Algorithm: Simulating a "Civilized" goto with break

1. Start.
2. Initialize a loop with an infinite range (e.g., while True).
3. Within the loop:
 - Perform tasks step by step.
 - Use a condition to determine if you need to "jump" to another section of the code.
 - When the condition is met, execute the break statement to exit the loop.
4. After the loop, perform the tasks that logically follow the loop exit, emulating a "goto" destination.
5. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 5: JAVA PROGRAM ON ARRAY

Write a JAVA Program to print the largest element in an array.

Algorithm: To Find the Largest Element in an Array

1. Start.
2. Input the size of the array (n) and the elements of the array.
3. Check if the array is empty:
 - If $n == 0$, output a message indicating the array is empty and terminate.
4. Initialize a variable largest with the first element of the array ($\text{largest} = \text{arr}[0]$).
5. Loop through the array from the second element (index 1) to the last element:
 - For each element, compare it with largest.
 - If the current element is greater than largest, update largest with the current element's value.
6. After the loop ends, largest will hold the maximum value in the array.
7. Output the value of largest.
8. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 6: JAVA PROGRAM ON MATRICES

Write a JAVA Program to Add Two Matrices.

Algorithm: To add two matrices

1. Start.
2. Input the dimensions of the matrices: rows and columns.
3. Verify if the dimensions of the two matrices are the same:
 - If they are not the same, output an error message and terminate the program.
4. Input the elements of the first matrix A.
5. Input the elements of the second matrix B.
6. Initialize an empty result matrix C of the same dimensions as A and B.
7. Loop through each row i (from 0 to rows-1):
 - Inside this loop, loop through each column j (from 0 to columns-1):
 - Add the corresponding elements: $C[i][j] = A[i][j] + B[i][j]$.
8. After all elements are processed, C will contain the sum of matrices A and B.
9. Output the resulting matrix C.
10. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 7: JAVA PROGRAM ON ARRAY

Write a JAVA program to find the largest element of an array and search the given key element from the array.

Algorithm: To search the given key element from the array

1. Start.
2. Input the size of the array (n).
3. Input the elements of the array (arr).
4. Input the key element to be searched (key).
5. Initialize a variable found to False.
6. Loop through the array:
 - For each element arr[i]:
 - Compare arr[i] with key.
 - If arr[i] == key:
 - Set found = True.
 - Output the position of the element (index i).
 - Break the loop.
7. After the loop, check the value of found:
 - If found == False, output "Key not found in the array."
8. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 8: JAVA PROGRAM ON TWO COMPLEX NUMBERS

Write and execute a JAVA program to add two complex numbers using Class.

Algorithm: To add two complex numbers using Class

1. Start.
2. Define a class Complex with the following:
 - Attributes real and imaginary to represent the real and imaginary parts of a complex number.
 - A constructor (`__init__`) to initialize a complex number with given values for the real and imaginary parts.
 - A method add to add two complex numbers. This method:
 - Takes another complex number as input.
 - Returns a new complex number whose real part is the sum of the real parts, and the imaginary part is the sum of the imaginary parts.
3. Create two objects of the Complex class to represent the two input complex numbers.
4. Call the add method on the first object, passing the second object as an argument.
5. Store the result in a new object and display its real and imaginary parts.
6. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 9: JAVA PROGRAM OBJECTS

Write and execute a JAVA program to create a class called Student with the following details as variables within it USN, NAME, BRANCH, PHONE, PERCENTAGE. Write a JAVA program to create n Student objects and print the USN, Name, Branch, Phone, and percentage of these objects with suitable headings.

Algorithm: By using object

1. Start.
2. Define a Student class with the following attributes:
 - String USN: to store the University Seat Number.
 - String NAME: to store the student's name.
 - String BRANCH: to store the branch of study.
 - String PHONE: to store the phone number.
 - double PERCENTAGE: to store the percentage of marks.
3. Add a constructor to the class to initialize the attributes of the Student object.
4. Add a method displayDetails() to print the details of the student in a formatted manner.
5. In the main method:
 - Input the number of students, n.
 - Create an array of Student objects of size n.
6. Use a loop to:
 - Input the details of each student (USN, NAME, BRANCH, PHONE, PERCENTAGE).
 - Create a Student object using the constructor and store it in the array.
7. Use another loop to:
 - Call the displayDetails() method for each Student object in the array to print their details.
8. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 10: JAVA PROGRAM ON METHOD AND CONSTRUCTOR OVERLOADING

Write and execute a JAVA program demonstrating Method overloading and Constructor overloading.

Algorithm. By using Method Overloading

1. Start.
2. Define a class, for example, Calculator.
3. Define multiple methods with the same name but different parameter lists:
 - A method add(int a, int b) to add two integers.
 - A method add(double a, double b) to add two doubles.
 - A method add(int a, int b, int c) to add three integers.
4. In the main method:
 - Create an instance of the Calculator class.
 - Call the add method with different numbers and types of parameters to demonstrate method overloading.
5. End.

Step-by-Step Algorithm for Constructor Overloading

1. Start.
2. Define a class, for example, Student.
3. Define multiple constructors with different parameter lists:
 - A default constructor Student() to initialize default values.
 - A constructor Student(String name, int age) to initialize name and age.
 - A constructor Student(String name, int age, String course) to initialize name, age, and course.
4. In the main method:
 - Create instances of the Student class using different constructors to demonstrate constructor overloading.
5. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 11: JAVA PROGRAM ON SUPER CLASS

Write a JAVA Program to demonstrate the uses of super. Design a super class called Staff with details as StaffId, Name, Salary. Extend this class by writing two subclasses namely Teaching (Expertise), and Technical (skills), Write a JAVA program to read and display staff details of all two categories.

Algorithm: By using super keyword

1. Start.
2. Define a superclass Staff with the following attributes:
 - StaffId: to store the staff ID.
 - Name: to store the name of the staff.
 - Salary: to store the salary of the staff.
 - Define a constructor to initialize these attributes.
 - Define a method displayDetails() to print the staff details.
3. Define two subclasses:
 - Teaching subclass, which extends Staff and has an additional attribute Expertise (to store the subject expertise).
 - Technical subclass, which extends Staff and has an additional attribute Skills (to store the technical skills).
4. In the subclasses, use super() to call the constructor of the Staff class to initialize common attributes (StaffId, Name, Salary).
 - For the Teaching class, add an attribute Expertise and implement the displayDetails() method to print all details (using super.displayDetails() to call the parent class method).
 - For the Technical class, add an attribute Skills and implement the displayDetails() method in a similar manner.
5. In the main method:
 - Read and input the details for both Teaching and Technical staff.
 - Create objects for both categories.
 - Call the displayDetails() method for both categories to display the staff details.
6. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 12: JAVA PROGRAM ON ABSTRACT CLASS

Write and execute a JAVA program to describe the abstract class called shape, which has three subclasses say triangle, rectangle and circle. Define one method area() in abstract class and override this area() in these three subclasses to calculate areas of triangle, rectangle and circle.

Algorithm: By using abstract class

1. Start.
2. Define the abstract class Shape:
 - Declare an abstract method area(), which will be overridden by subclasses to calculate the area of different shapes.
3. Define the subclass Triangle:
 - Inherit from the abstract class Shape.
 - Define the constructor to accept the base and height of the triangle.
 - Override the area() method to calculate the area of the triangle (formula: $0.5 * \text{base} * \text{height}$).
4. Define the subclass Rectangle:
 - Inherit from the abstract class Shape.
 - Define the constructor to accept the length and width of the rectangle.
 - Override the area() method to calculate the area of the rectangle (formula: $\text{length} * \text{width}$).
5. Define the subclass Circle:
 - Inherit from the abstract class Shape.
 - Define the constructor to accept the radius of the circle.
 - Override the area() method to calculate the area of the circle (formula: $\pi * \text{radius}^2$).
6. In the main method:
 - Create instances of the Triangle, Rectangle, and Circle classes by passing their respective parameters.
 - Call the area() method on each instance to calculate and display the area of each shape.
7. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 13: JAVA PROGRAM ON EXCEPTION

Write and execute a JAVA program to read two integers a and b. Compute a/b and print, when b is not zero. Raise an exception when b is equal to zero. Also demonstrate working of `ArrayIndexOutOfBoundsException`.

Algorithm: By using exception

Part 1: Handle Division by Zero (`ArithmeticException`)

1. Start.
2. Read two integers a and b from the user.
3. Use a try-catch block:
 - In the try block, check if b is not zero, and then compute a / b.
 - If b is zero, an `ArithmeticException` should be raised, and the catch block will handle this by printing an appropriate error message.
4. Print the result of a / b if b is non-zero.
5. If b is zero, print "Cannot divide by zero" in the catch block.
6. End.

Part 2: Demonstrate `ArrayIndexOutOfBoundsException`

7. Start another part of the program to demonstrate `ArrayIndexOutOfBoundsException`:
 - Create an array of fixed size (for example, of size 3).
 - Try to access an index that is out of bounds (e.g., index 5).
 - Use a try-catch block to catch the `ArrayIndexOutOfBoundsException` and print an error message when an invalid index is accessed.
8. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 14: JAVA PROGRAM ON EXCEPTIONS

Write and execute a JAVA program demonstrating finally block for handling exceptions.

Algorithm: By using finally block

1. Start.
2. Input Values:
 - Read inputs (e.g., two integers a and b) from the user.
3. Try-Catch Block:
 - Use a try block to attempt to perform an operation (e.g., division or any operation that might throw an exception).
 - If the operation fails (e.g., division by zero), catch the exception in the catch block and display an appropriate error message.
4. Finally Block:
 - Use a finally block that contains code to be executed regardless of whether an exception was thrown or not (e.g., closing resources, printing a message that indicates the end of the block).
 - The code inside the finally block will execute after the try and catch blocks, even if there is an exception.
5. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-

SESSION 15: JAVA PROGRAM ON INHERITANCE

Write and execute a JAVA program to implement multiple inheritance.

Algorithm: By using Multiple Inheritance Using Interfaces

1. Start.
2. Define the First Interface:
 - Define an interface Interface1 with some abstract methods (e.g., method1()).
3. Define the Second Interface:
 - Define another interface Interface2 with different abstract methods (e.g., method2()).
4. Define the Concrete Class:
 - Create a class ClassA that implements both Interface1 and Interface2.
 - The class should implement the methods from both interfaces (method1() and method2()).
5. Use the Class and Call Methods:
 - In the main method, create an instance of ClassA and call the methods method1() and method2() to demonstrate the functionality.
6. End.

INFERENCE:

DEBUG THE PROGRAMS AND INTRODUCE ERRORS IN THE PROGRAM AND ANALYSE THE ERRORS

-
-